

03_logreg

September 27, 2026

1 MA3622: Practical Machine Learning

variationalform https://variationalform.github.io/courses_PML/

Just Enough: progress at pace <https://github.com/variationalform>

Simon Shaw - <https://www.brunel.ac.uk/people/simon-shaw>.

2 Binary Classifiers - Logistic Regression

variationalform https://variationalform.github.io/courses_PML/

Just Enough: progress at pace <https://github.com/variationalform>

This work is licensed under CC BY-SA 4.0 (Attribution-ShareAlike 4.0 International)

Visit <http://creativecommons.org/licenses/by-sa/4.0/> to see the terms.

This document uses python

and also makes use of LaTeX

in Markdown

Google and possibly other search engines, both with and without AI enhancement, may have been used in the production of these notes.

AI tools such as Gemini, Claude, ChaptGPT etc may also have played a role in generating ideas and raw material for the following content.

Specific inclusions and sources are always referenced.

However, all planning and quality control is purely human, and my responsibility.

2.1 What this is about:

- The sigmoid function
- Binary classification, logistic regression.

2.2 Assigned Reading

For this worksheet you are recommended Chapter 3 of [MLF]:

- MLF: Machine Learning: A First Course for Engineers and Scientists, by Andreas Lindholm, Niklas Wahlström, Fredrik Lindsten, Thomas B. Schön. Cambridge University Press. <http://smlbook.org>.

There are also these useful references for coding:

- PT: python: <https://docs.python.org/3/tutorial>
- NP: numpy: <https://numpy.org/doc/stable/user/quickstart.html>
- MPL: matplotlib: <https://matplotlib.org>

2.3 Binary Classifier

The confusion matrix can be large or small. A particularly important case is the 2×2 case.

This type of classification arises when we are simply trying to decide ‘Yes’ or ‘No’, or ‘True’ or ‘False’, ‘Guilty’ or ‘Innocent’, ‘Diseased’ or ‘Healthy’ and so on.

There are only two classes: generically termed **positive** and **negative**.

Such a classifier is called a **Binary Classifier**, and the confusion matrix bears further discussion in this case.

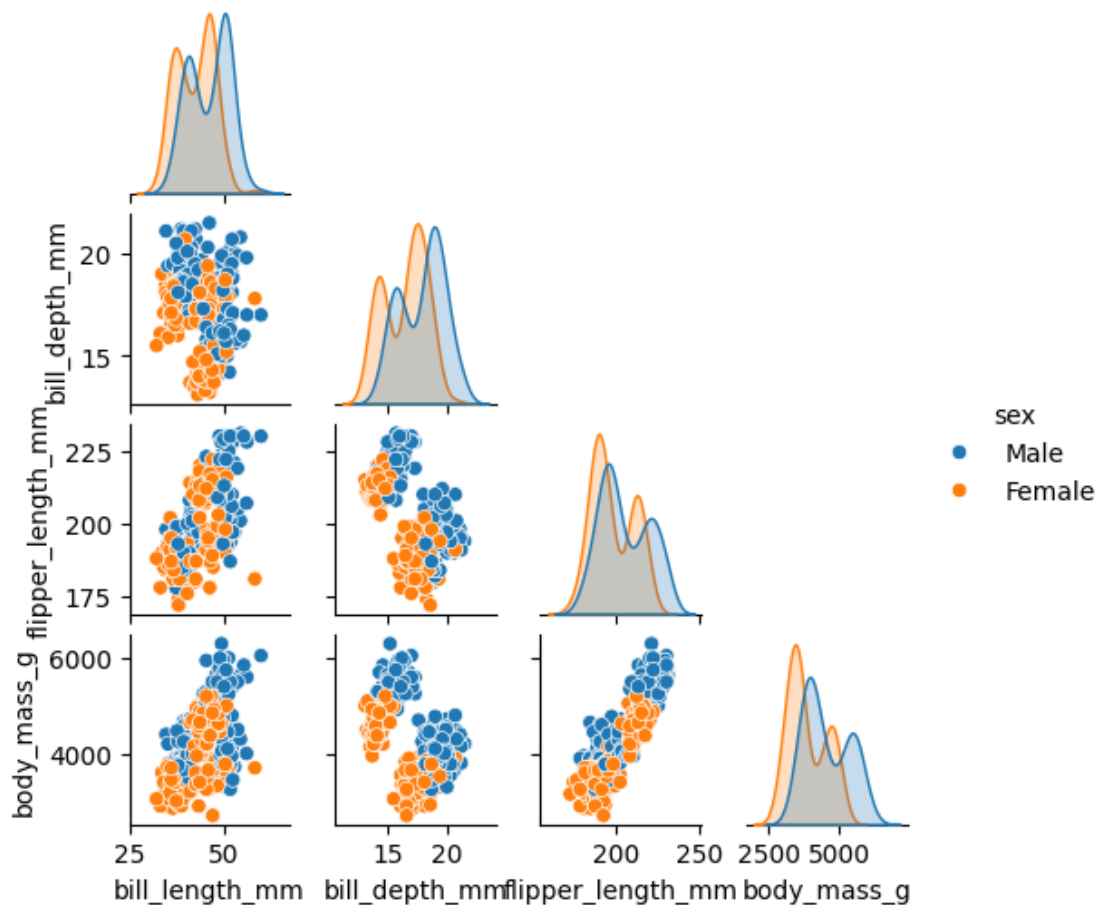
Let’s return to the penguin data and try to predict gender from the other physiological characteristics. This will be a binary classifier, because it will either predict ‘Female’ (**positive**) or ‘Not-Female’ (**negative**).

We’ll start by recalling the structure of the data frame, but first bring in our standard imports - with a new one: `linear_model` from `sklearn`

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import linear_model
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, \
    confusion_matrix, accuracy_score
from sklearn.metrics import ConfusionMatrixDisplay
```

```
[2]: dfp = sns.load_dataset('penguins')
dfp2 = dfp.dropna()
dfp2.isna().sum()
dfp2 = dfp2.reset_index(drop=True)
```

```
[3]: sns.pairplot(dfp2, corner=True, hue='sex', height=1.3);
```



It looks from this that *bill depth* and *body mass* should work well to separate out genders.

Even here, the term **Decision Boundary** might make some intuitive sense to you. What do you think it might mean?

Let's build the binary classifier using just these two columns, and get the confusion matrix just as before.

We'll do it all in one go, because we've seen all the steps before...

```
[4]: # extract the input/features X, and the output/labels y
X = dfp2.iloc[:, [3,5]].values
y = dfp2.iloc[:, 6].values
#print(X[0:4,:], '\n', y[0:4]) # optional, print a few, just to check
# bifurcate the data to get a 40% test set, and 60% training set
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.40)
# get scaling parameters from the training data,
scaler = StandardScaler()
scaler.fit(X_train)
```

```

# scale the training data
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

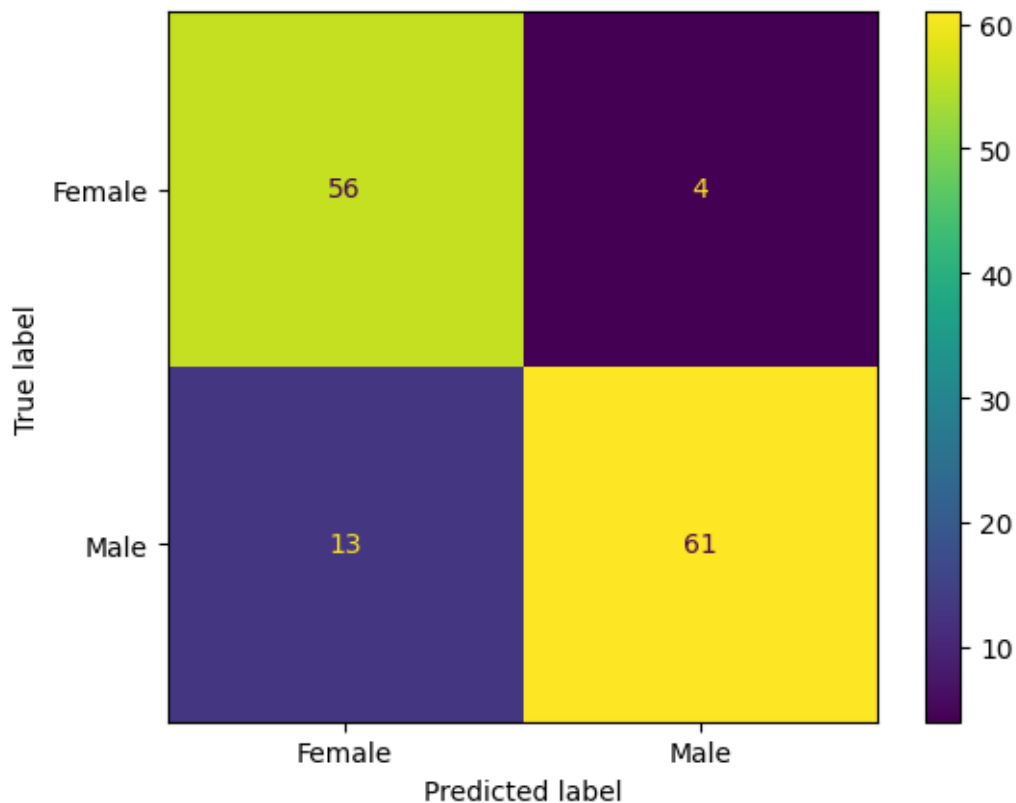
# now classify using k=2, p=1 - as before
classifier = KNeighborsClassifier(n_neighbors=2, p=1)
classifier.fit(X_train, y_train)
# and then make predictions from the test data
y_pred = classifier.predict(X_test)
# compare the predictions with the hold-out (test) set, y_test
cm = confusion_matrix(y_test, y_pred)

```

```

[5]: cmplot = ConfusionMatrixDisplay(cm, display_labels=classifier.classes_)
cmplot.plot();

```



2.4 True and False Positives and Negatives

There are a few particularly important pieces of terminology that are associated with binary classifiers.

- **TP, True Positives:** This is the number of test data points that are labelled **POSITIVE** for which the classifier correctly (**truly**) predicted them as **POSITIVE**.

- **FP, False Positives:** This is the number of test data points that are labelled **NEGATIVE** for which the classifier incorrectly (**falsely**) predicted them as **POSITIVE**.

Since we are regarding **Female** as **POSITIVE**, we can see these numbers in the first column above. Furthermore (second column),

- **FN, False Negatives.** This is the number of test data points that are labelled **POSITIVE** for which the classifier incorrectly (**falsely**) predicted them as **NEGATIVE**.
- **TN, True Negatives.** This is the number of test data points that are labelled **NEGATIVE** for which the classifier incorrectly (**truly**) predicted them as **NEGATIVE**.

Diagrammatically, for the binary confusion matrix we have:

$$\begin{array}{ccc}
 \text{target, or true} & + & \left(\begin{array}{cc} \text{TP} & \text{FN} \\ \text{FP} & \text{TN} \end{array} \right) \\
 \text{label/class} & - & \\
 & & + \quad - \\
 & & \text{output, or predicted} \\
 & & \text{label/class}
 \end{array}$$

We can access these numbers in code as follows:

```
[6]: TP = cm[0,0]; FP = cm[1,0]; FN = cm[0,1]; TN = cm[1,1]
print(f'{TP=:<d}, {FN=:<d}, {FP=:<d}, {TN=:<d}')
# note the concise f-string in this print statement
```

TP=56, FN=4, FP=13, TN=61

2.5 Related Formulae and Measures

Once we have these data we can compute a wide variety of different performance indicators. Here are the most commonly used, where we use P and N for the number of positives and negatives overall in the test set.

- **Prevalence:**

$$\text{Prevalence} = \frac{P}{P + N}$$

- **TPR: True Positive Rate, sensitivity, recall:**

$$\text{TPR} = \frac{\text{TP}}{P} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **TNR: True Negative Rate, specificity, selectivity:**

$$\text{TNR} = \frac{\text{TN}}{N} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

- **FPR: False Positive Rate:**

$$\text{FPR} = \frac{\text{FP}}{N} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- **FNR: False Negative Rate:**

$$\text{FNR} = \frac{\text{FN}}{\text{P}} = \frac{\text{FN}}{\text{FN} + \text{TP}}$$

- **Accuracy:**

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$$

- **Balanced Accuracy:**

$$\text{Balanced Accuracy} = \frac{1}{2} (\text{TPR} + \text{TNR})$$

- **PPV: Positive Predictive Value, *precision*:**

$$\text{PPV} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **NPV: Negative Predictive Value:**

$$\text{NPV} = \frac{\text{TN}}{\text{TN} + \text{FN}}$$

- **F1 score:** this is the *harmonic mean* of precision and sensitivity:

$$\text{F1} = \left(\frac{\left(\frac{\text{TP}}{\text{TP} + \text{FP}} \right)^{-1} + \left(\frac{\text{TP}}{\text{TP} + \text{FN}} \right)^{-1}}{2} \right)^{-1} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}$$

Let's calculate some of these

```
[7]: P=TP+FN
     N=FP+TN
     print('P (# positives) = ', P)
     print('N (# negatives) = ', N)
     print('TPR (recall) = ', TP/(TP+FN))
     print('PPV (precision) = ', TP/(TP+FP))
     print('F1 = ', 2*TP/(2*TP+FP+FN))
     print('Accuracy = ', (TP+TN)/(P+N))
```

```
P (# positives) = 60
N (# negatives) = 74
TPR (recall) = 0.9333333333333333
PPV (precision) = 0.8115942028985508
F1 = 0.8682170542635659
Accuracy = 0.8731343283582089
```

2.6 Straight Lines - Regression, Decision Boundaries

You've probably all met the idea of fitting the *best* straight line through a set of points. Some of you might have seen this in a second year module. If you haven't then don't worry, we'll cover everything we need in this module.

Suppose we have the following set of data and want to build a model that given x (a feature), predicts $y(x)$ (a label).

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \begin{pmatrix} 6 \\ 7 \end{pmatrix}, \begin{pmatrix} 7 \\ 5 \end{pmatrix}$$

Suppose further that we believe that the relationship is linear: $y = \theta_0 + \theta_1 x + \text{noise}$

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \begin{pmatrix} 4 \\ 3 \end{pmatrix}, \begin{pmatrix} 6 \\ 7 \end{pmatrix}, \begin{pmatrix} 7 \\ 5 \end{pmatrix}$$

This is all we need to do (there's a lot of maths tucked away in this):

```
[8]: X_vals = np.array([[0,1,2,4,6,7]]).T
     y_vals = np.array([[1,3,1,3,7,5]]).T
```

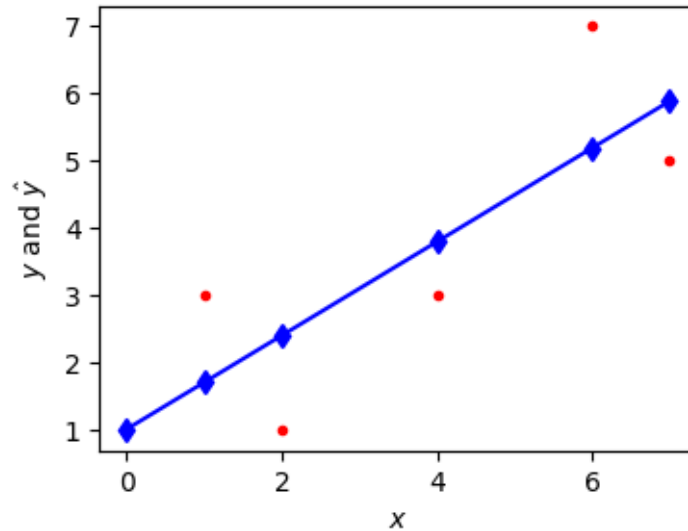
```
[9]: # standard regression
     reg_linear = linear_model.LinearRegression()
     reg_linear.fit(X_vals, y_vals)
     print('theta_1 (reg_coef_) = ', reg_linear.coef_)
     print('theta_0 (reg_intercept_) = ', reg_linear.intercept_)
```

```
theta_1 (reg_coef_) = [[0.69491525]]
theta_0 (reg_intercept_) = [1.01694915]
```

So $\hat{y} = 0.69491525x + 1.01694915$

We can make predictions of y , as $\hat{y}$, and plot our model like this:

```
[10]: # Make predictions from the fitted model
     y_pred = reg_linear.predict(X_vals)
     plt.figure(figsize=(4,3))
     plt.plot(X_vals,y_vals,'.r')
     plt.plot(X_vals,y_pred,'-b',marker='d')
     plt.xlabel(r'$x$'); plt.ylabel(r'$y$ and $\hat{y}$');
```



Straight lines can also give us **decision boundaries** ... and can lead to **classifiers**.

Here's a very important application of that idea.

2.7 Logistic Regression

This is a technique for binary classification that is actually based on ideas from regression. The idea is to use a straight line to create a **decision boundary** between the two classes.

The following logistic, or sigmoid, function is then used to create a switch from **off** to **on** as we cross that decision boundary.

This, for $a \in \mathbb{R}$, is the logistic function, often called the **sigmoid**:

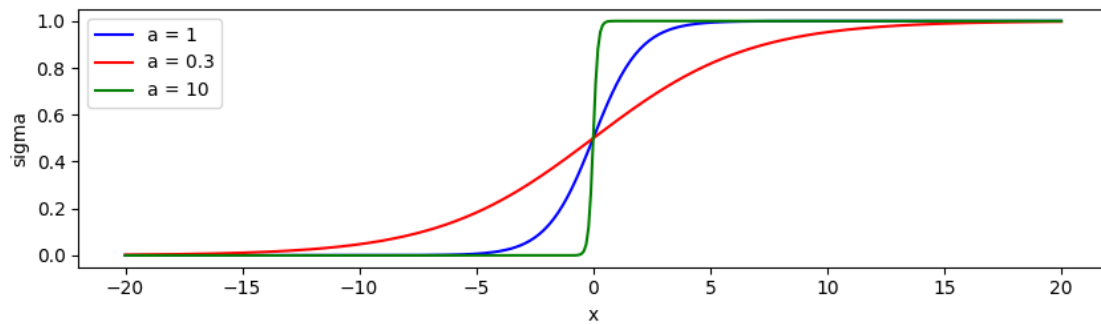
$$\sigma(x | a) = \frac{1}{1 + \exp(-ax)}, \quad \text{for } a > 0.$$

```
[11]: def sigma(x, a):
        return (1+np.exp(-a*x))**(-1)

x_vals = np.arange(-20, 20.1, 0.1)
y_vals_1 = sigma(x_vals, 1)
y_vals_03 = sigma(x_vals, 0.3)
y_vals_10 = sigma(x_vals, 10)

plt.figure(figsize=(10,4)); plt.gca().set_aspect(10)
plt.plot(x_vals, y_vals_1, color='blue', label='a = 1')
plt.plot(x_vals, y_vals_03, color='red', label='a = 0.3')
plt.plot(x_vals, y_vals_10, color='green', label='a = 10')
plt.xlabel('x'); plt.ylabel('sigma');
```

```
plt.legend()
plt.show()
```



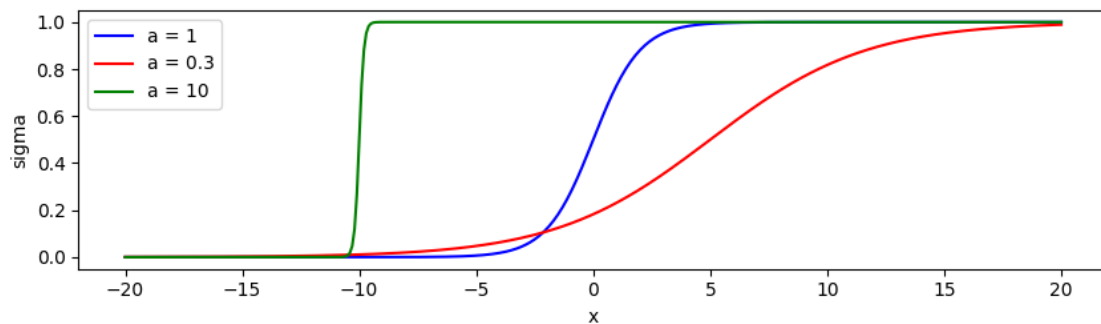
It can be scaled by the choice of a , and also translated by x_0 :

$$\sigma(x | a, x_0) = \frac{1}{1 + \exp(-a(x - x_0))}$$

```
[12]: def sigma(x, a, x0):
        return (1+np.exp(-a*(x-x0)))**(-1)

x_vals = np.arange(-20, 20.1, 0.1)
y_vals_0 = sigma(x_vals, 1, 0)
y_vals_5 = sigma(x_vals, 0.3, 5)
y_vals_m10 = sigma(x_vals, 10, -10)

plt.figure(figsize=(10,4)); plt.gca().set_aspect(10)
plt.plot(x_vals, y_vals_0, color='blue', label='a = 1')
plt.plot(x_vals, y_vals_5, color='red', label='a = 0.3')
plt.plot(x_vals, y_vals_m10, color='green', label='a = 10')
plt.xlabel('x'); plt.ylabel('sigma'); plt.legend();
```



In 2D we can write this kind of thing

$$\sigma(x_1, x_2 \mid a, b, c) = \frac{1}{1 + \exp(-(ax_1 + bx_2 + c))}$$

For example, with $a = -1$, $b = 1$ and $c = 0$ we have,

$$\sigma(x_1, x_2 \mid -1, 1, 0) = \frac{1}{1 + \exp(-(x_2 - x_1))}$$

Now, along the line $x_1 = x_2$ we have $\sigma(x_1, x_2 \mid -1, 1, 0) = 0.5$.

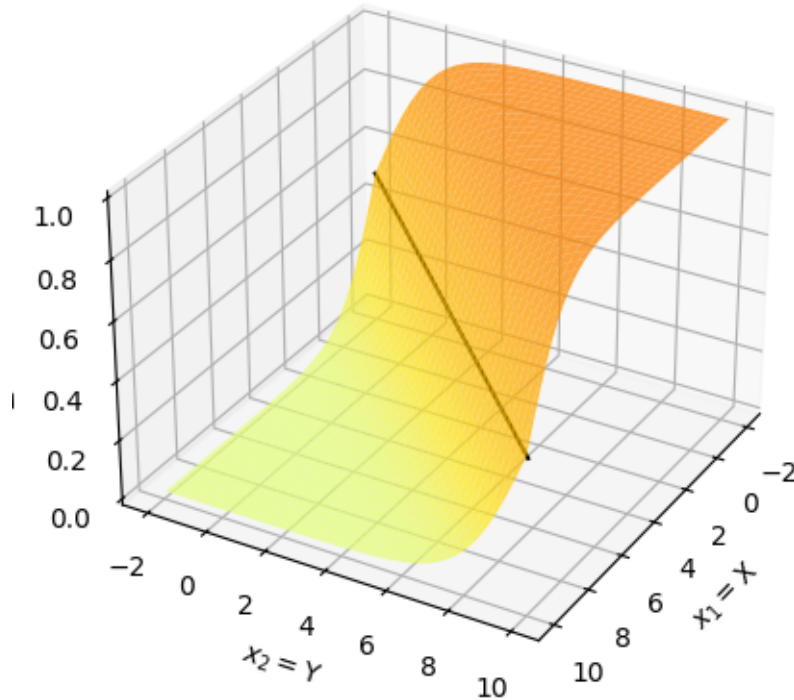
To one side of the line $\sigma \rightarrow 0$ and to the other side of the line $\sigma \rightarrow 1$. This means we have a switch, or a signal, that we can think of as being off on one side of the line and switched on if we are on the other side.

This is **binary classification**.

```
[13]: # Define dimensions
Nx, Ny, Nz = 10, 10, 1
X, Y, Z = np.meshgrid(np.arange(-2, Nx, 0.1),
                      np.arange(-2, Ny, 0.1), np.arange(Nz))
# Create sigmoid surface using the meshgrid
sigmoid = (1+np.exp(-(Y-X)))*(-1)

# we can plot the surface, and the decision boundary
# but we first load a colour map (confusingly also called cm!)
from matplotlib import cm

[14]: fig = plt.figure(figsize=(5,5)); ax = fig.add_subplot(projection='3d')
# plot sigmoid surface
ax.plot_surface(X[:, :, 0], Y[:, :, 0], sigmoid[:, :, 0],
               cmap=cm.Wistia, alpha=0.75)
ax.set(xlabel='$x_1=X$', ylabel='$x_2=Y$', zlabel='$x_3=Z$')
# plot decision boundary Y=X at height Z = 0.5
ax.plot([-2,10], [-2,10], [0.5,0.5], color='black')
ax.view_init(30, 30);
```



2.7.1 The Iris Data Set

We are going to illustrate the idea behind logistic regression using the Iris Data Set. See, for example, https://en.wikipedia.org/wiki/Iris_flower_data_set for the details.

This is very well known. Something that any aspiring Machine Learning engineer ought to be aware of.

It is not without controversy though. It was used by Fisher, and Fisher was associated with **eugenics**. You can read more about that here:

<https://www.nature.com/articles/s41437-020-00394-6>

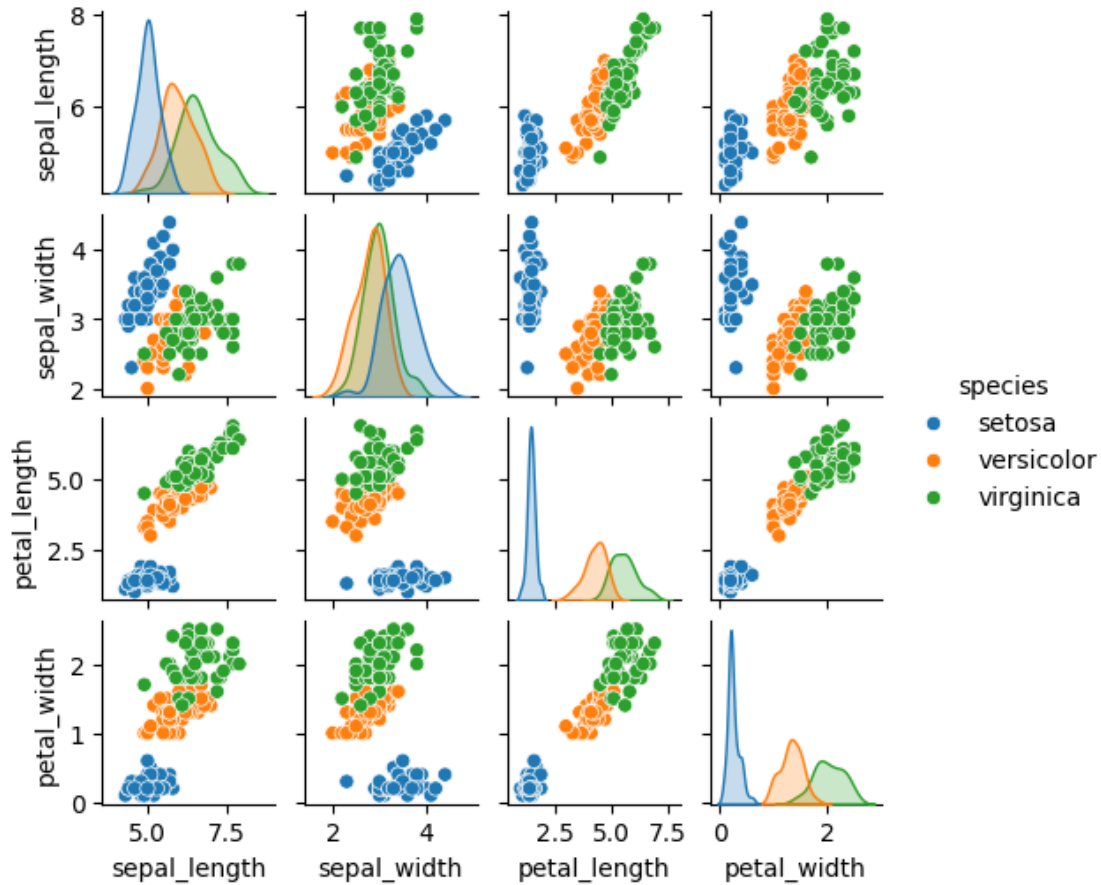
If this offends you then you can replace the use of the Iris data below with the penguins data we have been using.

```
[15]: # load it in and take a look
sns.get_dataset_names()
dfi = sns.load_dataset('iris')
dfi.head()
```

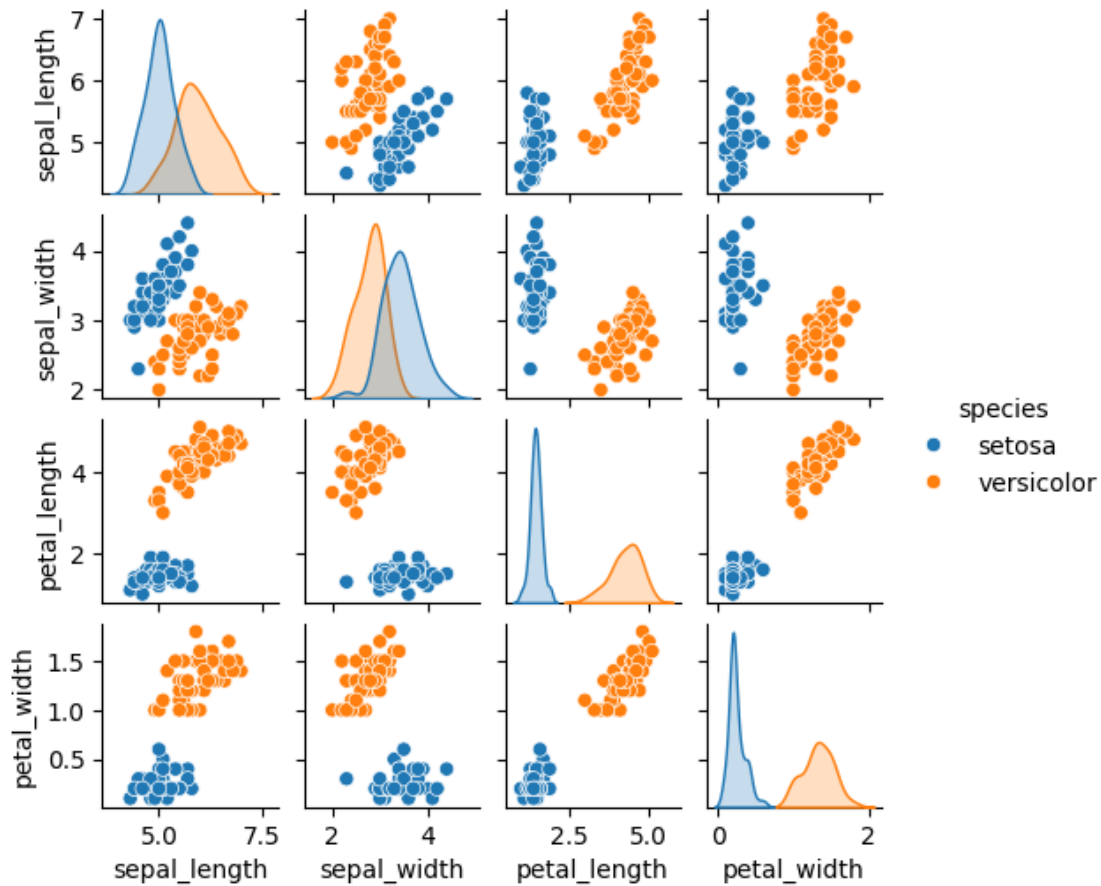
```
[15]:   sepal_length  sepal_width  petal_length  petal_width  species
0          5.1           3.5           1.4           0.2   setosa
1          4.9           3.0           1.4           0.2   setosa
```

2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
[16]: sns.pairplot(dfi, hue='species', height = 1.3);
```



```
[17]: # we want a binary classifier so we drop the virginica data
dfid = dfi[ (dfi['species'] != 'virginica') == True ]
sns.pairplot(dfid, hue='species', height = 1.3);
```



```
[18]: dfid.head()
```

```
[18]:   sepal_length  sepal_width  petal_length  petal_width  species
0         5.1         3.5         1.4         0.2    setosa
1         4.9         3.0         1.4         0.2    setosa
2         4.7         3.2         1.3         0.2    setosa
3         4.6         3.1         1.5         0.2    setosa
4         5.0         3.6         1.4         0.2    setosa
```

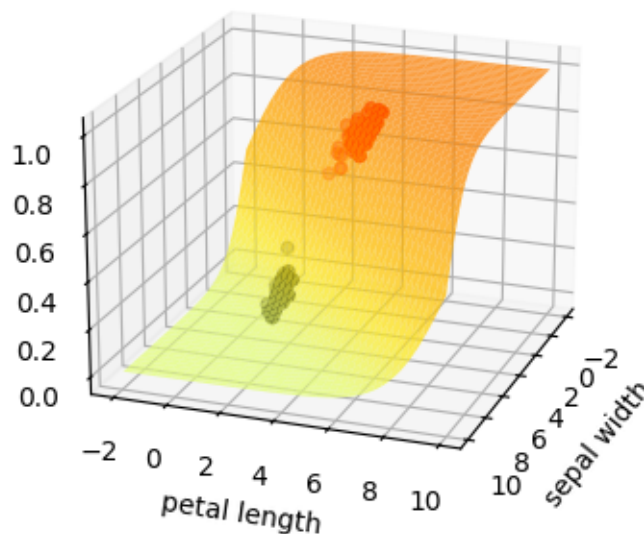
```
[19]: # let's use petal length and sepal width as our features
xall = dfid.iloc[:, [1, 2]].values
# and species as our label
yall = dfid.iloc[:, 4].values
print(xall[0:5, :], yall[0:5])
```

```
[[3.5 1.4]
 [3.  1.4]
 [3.2 1.3]
 [3.1 1.5]
```

```
[3.6 1.4]] ['setosa' 'setosa' 'setosa' 'setosa' 'setosa']
```

```
[20]: # select the setosa and versicolor feature rows
x_set = xall[yall == 'setosa',:]
x_ver = xall[yall == 'versicolor',:]
# set the vertical coordinate for the 3D surface plot
z_set = (1+np.exp(-(x_set[:,1]-x_set[:,0])))**(-1)
z_ver = (1+np.exp(-(x_ver[:,1]-x_ver[:,0])))**(-1)

[21]: fig = plt.figure(figsize=(4,4)) # some of this set up was done above
ax = fig.add_subplot(projection='3d')
ax.plot_surface(X[:, :, 0], Y[:, :, 0], sigmoid[:, :, 0], cmap=cm.Wistia,
               ↪alpha=0.75)
ax.scatter(x_set[:,0], x_set[:,1], z_set, c='black', marker='o')
ax.scatter(x_ver[:,0], x_ver[:,1], z_ver, c='red', marker='o')
ax.set_xlabel('sepal width'); ax.set_ylabel('petal length')
ax.set_zlabel('logistic values'); ax.view_init(20, 20); plt.show()
```



2.7.2 Discussion

What you see here is that the data are being separated by the sigmoid, or logistic, function's **ramp** from 0 to 1.

Note that we didn't actually do any machine learning - we just guessed the decision boundary!

The idea behind **logistic regression** is to use **training data to determine a line** $ax_1 + bx_2 + c = 0$ so that

$$\sigma(x_1, x_2 \mid a, b, c) = \frac{1}{1 + \exp(- (ax_1 + bx_2 + c))}$$

can be used as a classifier. The line is then a **decision boundary**.

For example, for input feature values x_1 and x_2 , and for a , b and c determined by the regression, we would classify as:

$$\begin{cases} \sigma(x_1, x_2 \mid a, b, c) \geq 0.5 & (x_1, x_2) \text{ indicate Class 1 (e.g. versicolor);} \\ \sigma(x_1, x_2 \mid a, b, c) < 0.5 & (x_1, x_2) \text{ indicate Class 2 (e.g. setosa);} \end{cases}$$

2.8 Final thoughts

No further theoretical details or explanations of Logistic Regression are required for this module, although **we will show how to implement it** and **you will be expected to be able to mimic that implementation** in similar contexts and for similar data sets.

You will meet Logistic Regression again, and learn more of its foundational mathematics, if you are enrolled on:

MA3627: Data Mining and AI for Big Data Analytics (opt M, comp MCS, FM, MDS)

You will have seen linear regression if you studied:

MA2619: Applied Statistics (not MCS)

HOWEVER: our module - MA3622 - is *self contained*.

2.9 Technical Notes, Production and Archiving

Ignore the material below. What follows is not relevant to the material being taught.

Production Workflow

- Finalise the notebook material above
- Clear and fresh run of entire notebook
- Create html slide show:
 - `jupyter nbconvert --to slides 11_regress.ipynb`
- Set `OUTPUTTING=1` below
- Comment out the display of web-sourced diagrams
- Clear and fresh run of entire notebook
- Comment back in the display of web-sourced diagrams
- Clear all cell output
- Set `OUTPUTTING=0` below
- Save
- git add, commit and push to FML
- copy PDF, HTML etc to web site
 - git add, commit and push
- rebuild binder

Some of this originated from

<https://stackoverflow.com/questions/38540326/save-html-of-a-jupyter-notebook-from-within-the-r>

These lines create a back up of the notebook. They can be ignored.

At some point this is better as a bash script outside of the notebook

```
[23]: %%%bash
NBROOTNAME=03_logreg
OUTPUTTING=1

if [ $OUTPUTTING -eq 1 ]; then
    jupyter nbconvert --to slides $NBROOTNAME.ipynb
    cp $NBROOTNAME.slides.html ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.
↪slides.html
    mv -f $NBROOTNAME.slides.html ../formats/slides/

    jupyter nbconvert --to pdf $NBROOTNAME.ipynb
    cp $NBROOTNAME.pdf ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.pdf
    mv -f $NBROOTNAME.pdf ../formats/pdf/

    jupyter nbconvert --to script $NBROOTNAME.ipynb
    cp $NBROOTNAME.py ../backups/$(date +"%m_%d_%Y-%H%M%S")_$NBROOTNAME.py
    mv -f $NBROOTNAME.py ../formats/py/
    echo; echo 'Finished generating html, pdf and py output versions'
else
    echo 'Not Generating html, pdf and py output versions'
fi
```

```
[NbConvertApp] Converting notebook 03_logreg.ipynb to slides
[NbConvertApp] WARNING | Alternative text is missing on 6 image(s).
[NbConvertApp] Writing 344361 bytes to 03_logreg.slides.html
[NbConvertApp] Converting notebook 03_logreg.ipynb to pdf
/Users/simon/offline/research/ALCAS/Nilocas_repos/NilocasUV/lib/python3.14/site-
packages/nbconvert/utils/pandoc.py:50: RuntimeWarning: You are using an
unsupported version of pandoc (2.7.2).
Your version must be at least (2.9.2) but less than (4.0.0).
Refer to https://pandoc.org/installing.html.
Continuing with doubts...
    check_pandoc_version()
[NbConvertApp] Writing 63811 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 94742 bytes to 03_logreg.pdf
```

```
[NbConvertApp] Converting notebook 03_logreg.ipynb to script  
[NbConvertApp] Writing 20261 bytes to 03_logreg.py
```

Finished generating html, pdf and py output versions